PlushPal

Product Requirements and Architecture Specification

Build baseline for iPhone, Android, macOS, and Windows

Version: 5.0

Status: Build baseline

Date: June 18, 2026

Privacy boundary: Local-first; sanitized search and approved cloud text only

Supersedes: Product Requirements and Architecture Specification v4.0

Architecture decision

Shared Flutter UI + Rust application core + local C++ inference. Capability-selected llama.cpp models are the baseline; curated search and approved cloud providers are separate, parent-controlled external modes.

Contents

1. Executive summary

PlushPal is a parent-configured conversational application that gives a physical plush character a locally synthesized voice. A parent creates a character, supplies a short voice reference they are authorized to use, selects an age band, and installs a device-compatible local conversation model. During normal use, microphone audio is transcribed locally, age-policy guardrails are enforced locally, the local model generates a response, the response is validated locally, and cloned speech is synthesized and played locally. A curated web lookup or approved production cloud model MAY be enabled separately by a parent under explicit external-processing disclosures.

The product is delivered as native applications for iPhone/iPad and Android, plus signed macOS and Windows applications that start a loopback-only service and open an embedded web UI in the user's default browser. The mobile applications and desktop host share the application core and speech engine. The desktop browser is a presentation client; it does not hold credentials, voice samples, or the authoritative database.

1.1 Product promise

PlushPal SHALL keep voice samples, raw microphone audio, speaker embeddings, synthesized audio, local model state, character records, settings, and authoritative conversation history within the user's device sandbox. Local-model turns SHALL require no network access. Curated web lookup MAY transmit a sanitized search query and retrieve public evidence. Approved cloud mode MAY transmit minimized text through a provider arrangement that permits the target age group and satisfies the required retention controls. No external mode may transmit raw audio, voice samples, embeddings, exact age, real names, local identifiers, or unrestricted history.

1.2 Delivery decision

This document approves the following baseline architecture:

- Flutter for the shared Android, iOS, and desktop-browser presentation layer.
- Rust for shared domain logic, application orchestration, age-policy enforcement, encryption, storage coordination, prompt construction, web retrieval, and conversation-provider abstraction.
- C++ inference libraries: ONNX Runtime/Sherpa-ONNX for local speech-to-text and text-to-speech plus llama.cpp for local GGUF conversation models, each exposed behind a stable C ABI.
- SQLite with SQLCipher for structured local data and encrypted files for voice assets.
- Native iOS and Android hosts for permissions, audio, key storage, model lifecycle, networking, and application lifecycle.
- A signed Rust desktop host that embeds the web assets, native speech engine, database, and a protected loopback HTTP/WebSocket service.

2. Document conventions

The terms SHALL and MUST identify mandatory release requirements. SHOULD identifies a strongly preferred requirement that may be waived through an architecture decision record. MAY identifies optional behavior. Each requirement has a stable identifier for implementation and test traceability.

Parent means the adult configuring the app and consenting to any external processing. **Child mode** means the restricted conversational experience. **Local** means within the selected device and its application-controlled storage. **Conversation provider** means the local llama.cpp runtime or a separately approved cloud

adapter. **Turn** means one child utterance and one character response. **Age policy** means the immutable application-controlled limits and safety behaviors derived from the parent-selected age band.

3. Goals, non-goals, and success measures

3.1 Goals

- Provide a warm, short, age-appropriate character conversation.
- Keep original and generated audio local.
- Reuse the application core and speech engine across iOS, Android, macOS, and Windows.
- Provide useful local conversation without a cloud LLM dependency on supported devices.
- Permit curated, sanitized web retrieval without exposing conversation history.
- Preserve a replaceable cloud-provider boundary for approved future production arrangements.
- Continue to provide character management and local playback when offline.
- Make provider, speech model, storage, and safety implementations replaceable.
- Fail safely and understandably under network, model, permission, and provider errors.

3.2 Non-goals for version 1

- PlushPal-hosted accounts, synchronization, analytics, or conversation services.
- Social networking, messaging between users, public voice sharing, or a character marketplace.
- Continuous background listening or wake-word operation.
- Guaranteed fully local LLM conversation on every device.
- Recording or retaining full microphone sessions by default.
- Clinical, emergency, educational assessment, or therapeutic claims.
- Child-directed purchasing, advertising, external links, or account creation.

3.3 Launch success measures

- At least 95% of supported-device turns complete without an application error in controlled beta testing.
- Median response playback begins within 5 seconds and the 95th percentile within 10 seconds on the reference network and device matrix.
- Local STT achieves the product-approved child-speech accuracy threshold defined by the evaluation corpus; initial go/no-go target is word error rate at or below 20% for supported English accents in a quiet room.
- No raw audio, voice sample, speaker embedding, credential, or full local database appears in captured network traffic.
- All mandatory safety, privacy, deletion, and parental-gate test cases pass before store submission.

4. Personas and operating modes

4.1 Parent administrator

The parent installs the application, passes the parental gate, selects an age band, reviews the capability assessment, installs a recommended local model, creates characters, authorizes voice use, manages external modes, reviews or deletes local history, and controls child-mode settings.

4.2 Child user

The child selects an enabled character, presses and holds or taps to speak, hears a response, interrupts playback if needed, and cannot access credentials, settings, external links, raw transcripts, purchases, or deletion controls.

4.3 Modes

- **Setup mode:** Adult-only onboarding and configuration.
- **Child mode:** Restricted conversation UI.
- **Private local mode:** Default conversational mode using local STT, local LLM, local policy enforcement, and local TTS.
- **Web-assisted local mode:** Local reasoning with curated external evidence obtained from sanitized searches.
- **Experimental cloud mode:** Supervised private-beta adapter using parent credentials; explicitly not approved for public child release.
- **Approved production cloud mode:** Available only after provider child-use, retention, contractual, safety, and privacy gates pass.
- **Offline play mode:** Character/profile management, previews, and deterministic local phrases when no compatible LLM is installed.

5. Privacy and data-processing contract

5.1 Data classification

Class	Examples	Storage	Network behavior
Restricted biometric/audio	Voice reference, speaker embedding, microphone buffers, generated PCM	Encrypted local files or memory only	Never transmitted
Restricted secret	Experimental cloud API key, database encryption key	OS credential store	Credential sent only to its configured provider
Sensitive child content	Transcript, local history, age band	Encrypted local database or session memory	Local by default; bounded text only in approved cloud mode
Configuration	Character alias, personality, provider/mode selection	Encrypted local database	Only non-identifying subset needed for approved external mode
Public retrieval	Sanitized search query, approved result URLs/content	Local bounded cache	Search query and normal HTTPS requests leave device
Operational	Model version, error code, duration metrics	Local bounded logs	No automatic telemetry in v1

5.2 Required disclosure

Before enabling curated search, the app SHALL disclose that sanitized search queries and ordinary page requests leave the device. Before enabling any cloud conversation mode, the app SHALL disclose that the selected provider receives text derived from the child's speech, bounded textual context, the device or gateway IP address, and provider-required metadata. It SHALL state that voice samples and raw audio are not sent. Experimental parent-BYOK mode SHALL be labeled supervised testing only. Production cloud mode SHALL remain unavailable until the provider eligibility record for the target age band is approved.

5.3 Data minimization

- The request SHALL use an age band, not a birth date.
- Real child and parent names SHALL NOT be included by the application.
- Character names SHALL be replaced with a generated alias when the display name appears person-identifying.
- The default local context SHALL contain no more than the active model's validated context budget. External requests SHALL contain no more than the last six completed turns, an optional local summary, and 2,500 input tokens.
- The application SHALL not send local identifiers, advertising identifiers, contact data, device names, filenames, file paths, voice embeddings, or raw audio.
- Cloud requests SHALL be stateless. Remote threads, conversations, files, assistants, provider-managed memory, and previous-response chaining are out of scope.
- Web search queries SHALL be locally rewritten to remove detected names, contact details, character data, and unrelated conversation context.

5.4 Retention and deletion

- Raw microphone buffers SHALL be released after STT and SHALL not be persisted by default.
- Stored history SHALL be disabled by default. Parents MAY enable local history and select 1 day, 7 days, 30 days, or manual deletion.
- Deleting a character SHALL cryptographically erase its local voice assets, embeddings, generated cache, and associated conversation records by deleting wrapped data keys and then removing files.
- “Delete all local data” SHALL be available behind the parental gate and complete without network access.
- Sensitive files SHALL be excluded from cloud/device backup where platform controls permit.

6. Functional requirements

6.1 Installation and onboarding

FR-ONB-001 The application SHALL support iOS/iPadOS, Android, macOS, and Windows versions in the supported-platform matrix.

FR-ONB-002 First launch SHALL present a parent-oriented explanation of local processing, optional search/cloud processing, model downloads, storage use, voice authorization, device limitations, and child safety limitations.

FR-ONB-003 The parent SHALL establish a parental gate before character creation. Mobile implementations SHALL use an adult challenge and MAY additionally use device authentication. Desktop implementations SHALL use a parent PIN protected by a memory-hard password derivation function.

FR-ONB-004 The parent SHALL select an age band: 4-5, 6-8, 9-12, or general family. Exact date of birth SHALL not be requested.

FR-ONB-005 Setup SHALL remain incomplete until speech models are available, at least one character exists, and either a compatible local conversation model is active or the application is explicitly left in offline play mode.

FR-ONB-006 The application SHALL assess memory class, free storage, architecture, available acceleration, operating-system support, and a bounded inference benchmark before recommending a local model tier.

FR-ONB-007 The parent SHALL review model download size, expected performance, battery impact, language, license, and quality tier before installation.

Acceptance: A new user can complete setup, restart the app, and enter child mode without recreating settings. A child-mode user cannot navigate back to parent controls without passing the gate.

6.2 Character and voice enrollment

FR-CHR-001 A parent SHALL create, edit, enable, disable, and delete up to 20 character profiles.

FR-CHR-002 A profile SHALL include a local identifier, display name, generated provider alias, age band override, personality traits from an approved vocabulary, optional parent-authored guidance, voice asset reference, creation time, and policy version.

FR-CHR-003 The enrollment flow SHALL accept WAV PCM input and MAY accept common mobile formats after local transcoding. The normalized internal format SHALL be mono 16-bit PCM at the speech model's required sample rate.

FR-CHR-004 The application SHALL validate duration, clipping, silence, signal-to-noise ratio, sample rate, and intelligibility before enrollment. The target reference duration is 15-30 seconds; invalid input SHALL receive actionable remediation guidance.

FR-CHR-005 The parent SHALL attest that they own or have permission to use the voice and SHALL be warned not to enroll a child or third party without appropriate authorization.

FR-CHR-006 Speaker embeddings SHALL be computed locally and encrypted. Implementations SHOULD avoid retaining the normalized reference after embedding only when the selected TTS engine does not require it and the parent chooses “remove original sample.”

FR-CHR-007 The app SHALL synthesize a local preview phrase and require parent approval before enabling the character.

Acceptance: Network inspection during import, normalization, embedding, preview, and deletion shows no audio or embedding transmission.

6.3 Conversation mode and provider configuration

FR-LLM-001 Private local mode SHALL be the default when a compatible local model is installed.

FR-LLM-002 Experimental cloud credentials SHALL be written directly to the platform credential store and SHALL never be returned to the Flutter UI after submission.

FR-LLM-003 Experimental cloud configuration SHALL validate credentials with the minimum provider request and display provider-authentication, quota, model-access, region, timeout, and policy errors separately.

FR-LLM-004 Local and cloud conversation adapters SHALL implement capability discovery, request generation, response parsing, cancellation, timeout/deadline classification, structured output, and safety-setting mapping where applicable.

FR-LLM-005 The private beta MAY include one OpenAI-compatible BYOK adapter for supervised testing. It SHALL not be represented as approved for public under-13 use. Custom endpoints SHALL be parent-only and clearly disclose that they receive conversation text and credentials.

FR-LLM-006 The app SHALL permit credential removal and mode/provider replacement without deleting local characters or local history.

FR-LLM-007 Automatic retries SHALL occur only for idempotent transient failures, use bounded exponential backoff, and never exceed two retries or the turn deadline.

FR-LLM-008 Production cloud providers SHALL be enabled only through an approved eligibility record containing provider/service, model family, permitted age bands, retention controls, region, contract/DPA status, safety evaluation, and expiry/review date.

FR-LLM-009 Gemini Developer API SHALL remain disabled while its terms prohibit clients directed toward or likely accessed by under-18 users. Any Google enterprise alternative requires a separately approved eligibility record.

FR-LLM-010 OpenAI production use for children under the applicable digital-consent age SHALL require an approved Zero Data Retention project and the application safeguards required by current under-18 API guidance.

Acceptance: The credential never appears in UI state snapshots, database contents, logs, crash reports, URLs, or exported diagnostics.

6.4 Conversation lifecycle

FR-CON-001 Child mode SHALL require an explicit tap or press-to-talk action. The microphone SHALL not remain active after the turn capture window.

FR-CON-002 The audio pipeline SHALL implement microphone permission handling, voice activity detection, maximum utterance duration, silence completion, resampling, echo control where available, and cancellation.

FR-CON-003 Maximum captured speech SHALL default to 20 seconds and be parent-configurable from 10 to 30 seconds.

FR-CON-004 STT SHALL execute locally. Partial transcripts MAY be displayed only in parent diagnostic mode; child mode SHALL not expose raw text by default.

FR-CON-005 Character routing SHALL use explicit selected-character state. Name matching MAY switch characters only when parent-enabled and shall escape regex metacharacters, normalize Unicode and case, resolve ambiguity, and never switch on a low-confidence match.

FR-CON-006 The conversation orchestrator SHALL maintain a local session state machine: Idle, Capturing, Transcribing, ScreeningInput, AwaitingProvider, ScreeningOutput, Synthesizing, Playing, Cancelled, and Failed.

FR-CON-007 A local generation request SHALL include the immutable system policy, age band, character alias, approved personality fields, bounded local context, and current transcript. External DTOs SHALL be compiled separately through explicit field allowlists.

FR-CON-008 Every local or cloud response SHALL conform to the structured response contract. Invalid, truncated, oversized, unsupported, ungrounded where search evidence is required, or policy-violating output SHALL not reach TTS.

FR-CON-009 TTS and voice cloning SHALL execute locally. Audio playback SHALL begin only after response validation.

FR-CON-010 A child SHALL be able to stop playback immediately. Starting a new capture SHALL cancel playback and any obsolete in-flight turn.

FR-CON-011 The app SHALL use a friendly local fallback for no speech, low-confidence transcription, incompatible device, model unavailable, insufficient memory, no network for an external mode, timeout, provider rejection, unsafe response, synthesis failure, and insufficient evidence.

FR-CON-012 The system SHALL serialize turns per session; late responses from cancelled turns SHALL be discarded using turn identifiers.

6.5 Safety behavior

FR-SAF-001 The system prompt SHALL be versioned, immutable in child mode, and instruct the model to produce short, warm, age-appropriate responses without solicitation, secrecy, dependency framing, sexual content, graphic violence, dangerous instructions, purchases, external contact, or claims of being human.

FR-SAF-002 “Remain in character” SHALL be subordinate to safety and truthfulness requirements.

FR-SAF-003 Input screening SHALL locally detect high-confidence categories requiring a blocked, supportive, or trusted-adult response. Keyword matching MAY support but SHALL NOT constitute the entire policy implementation.

FR-SAF-004 Output screening SHALL reject disallowed URLs, contact details, requests for personal data or secrets, explicit sexual content, graphic violence, dangerous instructions, manipulative relationship language, and output exceeding configured sentence or character limits.

FR-SAF-005 Cloud providers SHOULD be configured with available safety controls. Provider moderation calls SHALL follow the same eligibility, disclosure, retention, and minimization rules as generation calls. The generative model SHALL never be the only safety decision-maker.

FR-SAF-006 Distress, abuse, self-harm, or emergency language SHALL trigger a short non-diagnostic response encouraging the child to contact a nearby trusted adult. The app SHALL not represent itself as an emergency service.

FR-SAF-007 The initial release SHALL support only languages with evaluated STT, TTS, prompt, and safety behavior. Unsupported-language input SHALL receive a local fallback.

FR-SAF-008 Every released policy or prompt change SHALL pass the safety regression corpus and be versioned in the character/session record.

6.6 Parent controls and local history

FR-PAR-001 Parent mode SHALL display provider status, estimated model storage, enabled characters, history policy, data deletion, diagnostic export, and privacy disclosures.

FR-PAR-002 Parents MAY review locally stored transcripts when history is enabled. Audio SHALL not be stored as part of history.

FR-PAR-005 Parent mode SHALL show whether the active turn mode is local, web-assisted, experimental cloud, or approved production cloud, and SHALL summarize what data leaves the device for that mode.

FR-PAR-003 Diagnostic export SHALL be opt-in, locally generated, and redact transcripts, credentials, voice data, personal names, file paths, and persistent identifiers by default.

FR-PAR-004 Parents SHALL be able to reset the parent PIN through device-owner authentication where available; otherwise reset SHALL delete protected application data.

6.7 Model management

FR-MOD-001 Speech models SHALL be signed or hash-verified before activation.

FR-MOD-002 The app SHALL display download size, installed size, progress, cancellation, retry, and required free space.

FR-MOD-003 A failed update SHALL preserve the last known-good model. Model activation SHALL be atomic.

FR-MOD-004 Model manifests SHALL include model ID, semantic version, engine compatibility, language, checksum, license attribution, download source, and minimum device capabilities.

FR-MOD-005 Android MAY use Play Asset Delivery or an approved first-party distribution path. iOS SHALL use an App Store-compatible resource strategy. Desktop installers MAY bundle baseline models or download verified assets after parent approval.

FR-MOD-006 The signed recommendation manifest SHALL define capability tiers, supported runtimes, expected memory/storage, model hash, license, evaluation version, and policy compatibility.

FR-MOD-007 Initial candidate tiers are Qwen3 1.7B Q4 for standard capable mobile devices and Qwen3 4B Q4 for enhanced mobile/desktop devices, subject to the feasibility gate. Model names are implementation candidates, not unconditional release guarantees.

FR-MOD-008 The application SHALL preserve the last known-good model until a new model passes integrity, compatibility, load, structured-output, policy, and bounded performance self-tests.

6.8 Local conversation and curated web retrieval

FR-LOC-001 The conversation-provider abstraction SHALL support llama.cpp local generation as the baseline without changing conversation orchestration.

FR-LOC-002 iOS MAY implement Apple Foundation Models and Android MAY implement system GenAI as optional platform adapters where device, locale, policy, and acceptable-use requirements pass. Desktop MAY support a parent-configured Ollama/OpenAI-compatible loopback adapter.

FR-LOC-003 Capability detection SHALL be runtime-based. The UI SHALL not promise local conversation when the model, locale, device, or operating-system feature is unavailable.

FR-LOC-004 A local provider SHALL receive the same prompt policy and pass the same output validation as a cloud provider.

FR-LOC-005 Curated search SHALL be invoked only by the Rust orchestrator after age-policy authorization and query sanitization. The model SHALL not directly execute URLs, scripts, or arbitrary tools.

FR-LOC-006 Search fetching SHALL require HTTPS, validate redirects, block private/loopback/link-local addresses, restrict content types and sizes, remove active content, label evidence untrusted, and cap sources and cache lifetime.

FR-LOC-007 Web-assisted answers SHALL identify supporting source records internally and refuse or use a safe uncertainty response when approved evidence is insufficient or contradictory.

7. Non-functional requirements

7.1 Performance and resource budgets

ID	Requirement
NFR-PERF-001	UI actions SHALL acknowledge within 100 ms; operations longer than 300 ms SHALL expose progress or state.
NFR-PERF-002	Local STT SHOULD complete within 1.5x utterance duration on minimum supported devices and within 0.75x on reference devices.
NFR-PERF-003	TTS SHOULD synthesize faster than real time on reference devices and no slower than 1.5x real time on minimum devices.
NFR-PERF-004	Local generation and each external call SHALL have separate bounded deadlines. Defaults are 30 seconds for local first-token readiness and 12 seconds for cloud/search calls, with safe cancellation on expiry.
NFR-PERF-005	Capability tiers SHALL define measured memory headroom rather than one universal RAM number. A model SHALL load only when benchmarked peak use remains below the platform-tested termination threshold with at least 20% working-memory headroom.
NFR-PERF-006	A single turn SHALL not retain raw PCM after completion. Reusable buffers SHALL be bounded.
NFR-PERF-007	Thermal throttling and low-memory conditions SHALL degrade gracefully by disabling previews, reducing concurrency, or asking the user to retry.

7.2 Reliability

- Application state writes SHALL be transactional.
- A crash or forced termination during recording, local generation, cloud response, search, synthesis, model download, or deletion SHALL not corrupt the database or expose plaintext assets.
- The local desktop host SHALL select another port if the default is unavailable and SHALL never bind to a non-loopback interface.

- Model loading, generation, provider, search, speech, and download operations SHALL support cancellation and deadlines.
- Database migrations SHALL be versioned, reversible where possible, and tested from every supported released schema.

7.3 Security

- All cloud-provider and search traffic SHALL use HTTPS with normal platform certificate validation. Insecure HTTP endpoints SHALL be rejected except an explicitly enabled loopback local provider.
- At-rest files SHALL use authenticated encryption. The database SHALL use SQLCipher or equivalent full-database encryption.
- Data-encryption keys SHALL be random per installation or profile and wrapped by platform-protected keys.
- Secrets SHALL not cross the Dart boundary after initial credential submission.
- Release artifacts, model manifests, and updates SHALL be signed.
- Dependencies SHALL be pinned, scanned, licensed, and represented in an SBOM.
- Compiler hardening, stack protection, address-space randomization, and release symbol separation SHALL be enabled where supported.
- Sensitive memory SHOULD be zeroized when practical; no design SHALL claim guaranteed zeroization across managed runtimes.

7.4 Accessibility and usability

- Mobile applications SHALL meet WCAG 2.2 AA-equivalent platform expectations for contrast, labels, focus order, dynamic text, reduced motion, and screen readers.
- Every audio-only state SHALL have a visual equivalent for parents; child mode MAY keep transcripts hidden while still exposing state and error cues accessibly.
- Controls SHALL meet platform touch-target guidance.
- The application SHALL support interruption by calls, audio-route changes, headphones, Bluetooth, and device lock without losing data integrity.

7.5 Compatibility baseline

The final minimum OS versions SHALL be selected after the inference spike. Planning baseline: iOS/iPadOS 17 or later, Android API 29 or later with arm64-v8a as the primary ABI, macOS 13 or later on Apple Silicon and supported Intel where benchmarks pass, and Windows 11 x64/ARM64 where inference dependencies pass. Unsupported devices SHALL fail capability checks before downloading models.

8. System architecture

8.1 Logical components

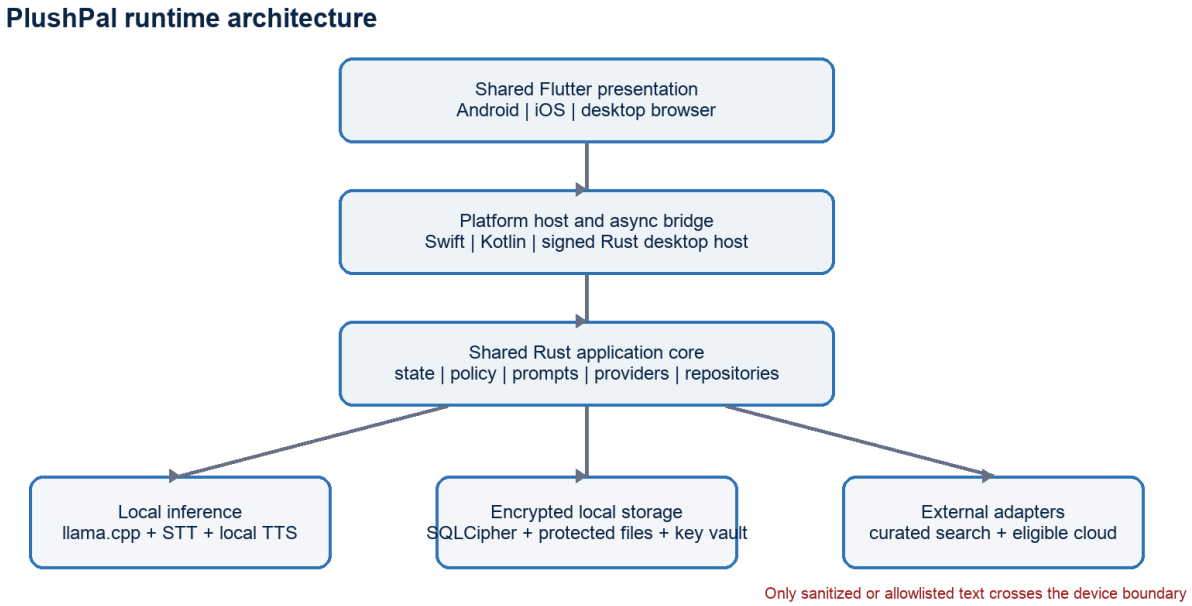

Figure 1. Shared runtime boundaries and controlled external text boundary

1. **Presentation:** Shared Flutter screens, navigation, accessible controls, and state rendering. It contains no provider secrets and no direct database access.
2. **Platform host:** Swift/Objective-C on iOS, Kotlin/Java on Android, and Rust executable on desktop. It owns lifecycle, permissions, credential storage, audio devices, secure networking hooks, and OS integrations.
3. **Application core:** Rust library implementing use cases, session state machine, policy, prompt compilation, provider interfaces, encrypted repositories, migrations, and typed events.
4. **Speech engine:** C++ library implementing model loading, audio normalization, VAD integration, STT, speaker preparation, and TTS.
5. **Local inference:** llama.cpp adapter implementing the common conversation-provider contract, verified model loading, bounded generation, structured output, and cancellation.
6. **Cloud adapters:** Native HTTPS clients implementing the same contract only for providers marked eligible by the release policy registry.
7. **Curated-search adapter:** Query sanitizer, HTTPS fetcher, private-address blocker, extractor, evidence normalizer, and bounded encrypted cache.
8. **Storage adapters:** SQLCipher repository, encrypted asset store, platform key vault, model store, and search cache.
9. **Desktop gateway:** Loopback HTTP/WebSocket API serving embedded Flutter assets and mapping authenticated local requests to application-core commands.

8.2 Dependency rules

- Presentation depends only on generated application-client interfaces and view models.
- Application core depends on traits/interfaces, not platform SDKs or concrete providers.
- Platform adapters depend on the core contracts.

- Speech C++ does not access networking, UI, provider credentials, or SQL.
- Local providers receive a bounded internal conversation object. External cloud and search adapters receive separate allowlisted DTOs and cannot query local repositories directly.
- Safety validation is invoked by orchestration and cannot be bypassed by a provider adapter.

8.3 Mobile deployment

On iOS and Android, Flutter communicates through a generated plugin boundary to the native host. The host calls the Rust core through a stable C-compatible interface. The Rust core calls the speech engine through a separate C ABI. Long operations return job handles and emit typed progress events; they do not block the UI thread.

8.4 Desktop deployment

The installer contains a signed native host, embedded web assets, migration resources, licenses, and either a baseline speech model or a verified model downloader. The native host starts on `127.0.0.1` using an ephemeral port, generates a 256-bit launch secret, opens the browser to a URL containing a one-time bootstrap fragment, exchanges it for a Secure/HttpOnly/SameSite session cookie where browser behavior permits, clears the bootstrap value, and rejects requests without a valid session.

The host SHALL validate `Host` and `Origin`, disable wildcard CORS, set a restrictive Content Security Policy, reject cross-site WebSocket upgrades, cap request bodies, rate-limit authentication failures, and shut down after the final UI session closes or a configured idle timeout expires. Static assets SHALL be embedded; end users SHALL not run Node, npm, or a development server.

9. Core contracts

9.1 Stable native boundary

The cross-language ABI SHALL use opaque handles, fixed-width integer types, caller/callee ownership documentation, UTF-8 byte spans, typed result codes, and explicit free functions. C++ virtual classes, `std::string`, `std::vector`, Rust layout types, and exceptions SHALL NOT cross the ABI.

Representative contract:

```
typedef struct pp_engine pp_engine_t;
typedef uint64_t pp_job_id_t;

typedef enum {
  PP_OK = 0,
  PP_INVALID_ARGUMENT,
  PP_NOT_READY,
  PP_PERMISSION_DENIED,
  PP_CANCELLED,
  PP_TIMEOUT,
  PP_PROVIDER_ERROR,
  PP_MODEL_ERROR,
  PP_STORAGE_ERROR,
  PP_POLICY_BLOCKED,
  PP_INTERNAL_ERROR
} pp_status_t;

pp_status_t pp_engine_create(const pp_engine_config_t*, pp_engine_t** out);
pp_status_t pp_engine_start_turn(pp_engine_t*, const pp_turn_request_t*, pp_job_id_t* out);
pp_status_t pp_engine_cancel_job(pp_engine_t*, pp_job_id_t);
```

```
pp_status_t pp_engine_poll_event(pp_engine_t*, pp_event_t* out);
void pp_event_release(pp_event_t*);
void pp_engine_destroy(pp_engine_t*);
```

Every ABI SHALL carry a version and structure-size field so fields can be added compatibly.

9.2 Conversation-provider contract

```
trait ConversationProvider: Send + Sync {
    fn capabilities(&self) -> ConversationCapabilities;
    async fn prepare(&self, config: ProviderConfig) -> Result<ProviderIdentity, ProviderError>;
    async fn generate(
        &self,
        request: BoundedConversationRequest,
        deadline: Instant,
        cancel: CancellationToken,
    ) -> Result<StructuredCharacterResponse, ProviderError>;
}
```

Provider errors SHALL be normalized into authentication, permission/model access, quota, rate limit, content rejection, network unavailable, TLS, timeout, malformed response, incompatible device, model unavailable, memory pressure, cancellation, and internal categories.

The orchestrator SHALL derive an external `MinimizedConversationRequest` only after provider eligibility, parental consent, age policy, and data minimization checks pass. Local providers SHALL never be forced through a network-shaped contract.

9.3 Structured provider response

```
{
  "schema_version": 1,
  "speech": "The moon looks like it follows us because it is very far away!",
  "safety": "normal",
  "suggest_trusted_adult": false
}
```

The core SHALL ignore unknown fields, enforce types and limits, reject missing required fields, strip unsupported markup, and never interpret model-provided instructions as executable commands.

9.4 Desktop local API

The loopback API SHALL be versioned under `/api/v1`. Commands include status, device-capability assessment, onboarding, character management, conversation-mode and provider configuration, model management, curated-search controls, session start/stop, audio capture control, history management, and deletion. Binary audio SHALL flow only between the browser UI and local host when required for desktop microphone capture; it SHALL not be exposed outside loopback and SHALL not be persisted before local policy permits.

WebSocket events SHALL include job ID, session ID, monotonic sequence, event type, safe display message, progress, and optional non-sensitive payload. Events SHALL never contain credentials or unrestricted filesystem paths.

10. Data architecture

10.1 Core entities

- `Installation`: schema version, policy version, locale, created time.
- `ParentSettings`: age band, gate verifier, history policy, retention period, external-processing acknowledgements, feature flags.
- `Character`: ID, display name, provider alias, personality, guidance, enabled state, language, voice asset ID, timestamps.
- `VoiceAsset`: ID, encrypted reference location, encrypted embedding location, engine/model versions, quality metrics, wrapped key ID.
- `ProviderConfig`: ID, provider type, endpoint, model, credential reference, safety configuration, validation status. No secret value.
- `ConversationConfig`: selected mode, local model ID, cloud provider ID, search setting, fallback behavior, last capability assessment.
- `ProviderEligibility`: provider/model, region, minimum policy version, retention requirement, release channel, legal/security approval state, expiry.
- `ConversationSession`: ID, character ID, local-only timestamps, policy/prompt/model versions.
- `ConversationTurn`: ID, session ID, local transcript and response when history enabled, outcome category, durations. No audio.
- `ModelInstallation`: model manifest, path, checksum, activation state.
- `SearchCacheEntry`: normalized query hash, bounded evidence records, source URLs, retrieval/expiry timestamps, policy version.
- `AuditEvent`: local bounded record of parent actions and non-content error categories.

10.2 Storage layout

```
app-data/
  database/plushpal.db
  assets/voices/<opaque-id>.bin
  assets/embeddings/<opaque-id>.bin
  cache/generated/<opaque-id>.bin
  cache/search/<opaque-id>.bin
  models/<model-id>/<version>/
  logs/local-redacted/
```

Filenames SHALL be opaque identifiers. User-provided filenames SHALL not be retained unless explicitly displayed only in parent mode. Temporary plaintext files SHALL not be used; transcoding and inference SHOULD stream through memory or encrypted temporary storage.

10.3 Key management

An installation master key SHALL be generated locally. On iOS it SHALL be protected by Keychain access controls; on Android by Android Keystore-backed wrapping; on macOS by Keychain; and on Windows by DPAPI/Credential Manager. Separate derived or random data keys SHOULD protect database, voice assets, embeddings, and caches. Character deletion SHALL destroy the character's wrapped asset keys before file removal.

11. Speech and audio architecture

11.1 Capture

Platform audio adapters SHALL negotiate supported formats and convert to the engine format. Audio callbacks SHALL avoid allocation, logging, database work, and network access. A bounded ring buffer SHALL bridge real-time capture to worker threads.

11.2 STT

The engine SHALL expose model load/unload, streaming or utterance transcription, confidence where supported, language detection where approved, and cancellation. The chosen model SHALL be benchmarked on child speech, not selected solely from adult speech benchmarks.

11.3 TTS and cloning

The engine SHALL separate speaker preparation from synthesis so embeddings can be reused locally. Synthesis input SHALL be validated plain text. Maximum output length, memory, and generation duration SHALL be bounded. The release process SHALL verify model and dataset licensing permits commercial redistribution and voice cloning use.

11.4 Audio session behavior

iOS SHALL configure `AVAudioSession` and `AVAudioEngine` for record/playback, interruptions, route changes, and Bluetooth policy. Android SHALL use appropriate `AudioRecord` and `AudioTrack` modes with audio focus. Desktop SHALL enumerate devices without persisting device names unless necessary for local settings.

12. Prompt and conversation design

12.1 Prompt layers

10. Versioned safety, identity, and tool-use system instructions.
11. Immutable age-band communication, privacy, search, and escalation rules.
12. Character alias and approved personality fields.
13. Bounded recent turns with delimiters and role labels.
14. Current transcript treated strictly as untrusted child content.
15. Structured response schema and maximum length.

Parent-authored character guidance SHALL be length-limited, screened, and placed below immutable safety instructions. It SHALL not be able to disable safety, request hidden chain-of-thought, or cause disclosure of system instructions.

12.2 Context policy

The context compiler SHALL select completed turns only, remove duplicate or cancelled turns, enforce token and turn limits, and avoid remote provider session IDs. Local encrypted history is authoritative. Local providers MAY receive the bounded active context; cloud providers receive a separately minimized

projection. When history is disabled, completed turns MAY remain in encrypted memory for the active session and SHALL be discarded when the session ends.

12.3 Response constraints

Default response limits are three short sentences, 450 characters, no Markdown, no URLs, no phone numbers, no requests for identifying information, and no first-person claims of being a real person. Provider output SHALL be normalized and validated before synthesis.

13. Threat model and controls

Threat	Primary controls
Extract API credential from UI/database	OS credential store; secret references; no Dart return path; redacted logs
Voice asset exfiltration	No upload code path; encrypted assets; provider DTO excludes binary fields; network tests
Prompt injection by child transcript	Role-delimited untrusted input; immutable policy; structured output; local validation
Prompt injection in retrieved web content	Search remains a core-controlled tool; active content removed; evidence marked untrusted; citations and grounding validated
Unsafe or manipulative response	Provider controls; output policy; bounded text; fallback; regression corpus
Ineligible cloud provider enabled	Signed eligibility registry; release-channel policy; expiry; fail closed; parent disclosure does not override provider terms
Malicious custom endpoint	Parent gate and disclosure; HTTPS requirement; allow loopback HTTP only; endpoint validation
Desktop cross-site attack	Loopback bind; launch token; session cookie; Origin/Host validation; strict CORS/CSP
DNS rebinding	Host allowlist containing only loopback literals; reject unexpected Host headers
Local database theft	SQLCipher; platform-wrapped key; backup exclusion; parent-gated export
Model tampering	Signed/hash-verified manifest; atomic activation; rollback
Dependency compromise	Lockfiles; SBOM; signature verification; vulnerability scanning; reviewed updates
Excessive retention	Default no history; configurable expiry; startup cleanup; delete-all and crypto erasure
Voice impersonation misuse	Adult gate; rights attestation; local-only asset design; no public export in v1

Security review SHALL include mobile static/dynamic analysis, desktop local-service penetration testing, network capture, filesystem inspection, backup inspection, secret scanning, and adversarial prompt testing.

14. Platform-specific design

14.1 iOS and iPadOS

- Package Rust and C++ components as compatible static libraries/XCFrameworks with simulator slices used only for development.
- Use Keychain for secrets and wrapped keys; use file-protection classes for sensitive files.
- Declare microphone purpose accurately and avoid requesting permission until parent setup or first use.
- Exclude voice assets, credentials, and history from iCloud backup where required by the privacy promise.
- Use the common llama.cpp runtime for the baseline local model where feasibility passes. Apple Foundation Models MAY be an optional Swift-native adapter, never a mandatory dependency.
- Comply with Kids Category restrictions if distributed in that category, including parental gates and third-party data disclosures.

14.2 Android

- Prioritize arm64-v8a. Additional ABIs require demonstrated device need and passing model benchmarks.
- Use Android Keystore for key wrapping and app-internal storage for protected files.
- Use foreground UI-bound microphone capture only; no background listening service.
- Use Play Asset Delivery or an approved asset mechanism for non-executable model weights after confirming current store rules.
- Use the common llama.cpp runtime for the baseline local model where feasibility passes. System GenAI/AICore MAY be an optional capability-gated adapter.

14.3 macOS and Windows

- Deliver signed/notarized installers and native uninstaller behavior.
- Store data in OS-standard per-user application directories.
- Use macOS Keychain and Windows DPAPI/Credential Manager.
- Bind the service only to IPv4 and IPv6 loopback as explicitly tested; never `0.0.0.0`.
- Embed production web assets and use no runtime package manager.
- Support optional Ollama/OpenAI-compatible localhost providers behind parent configuration.
- Ship the common llama.cpp runtime as the default local provider on supported desktop tiers; localhost adapters remain optional advanced choices.

15. Repository and build architecture

```
plushpal/
  apps/
    flutter_ui/
    ios_host/
    android_host/
    desktop_host/
  crates/
    core_domain/
    application/
    safety_policy/
    prompt_compiler/
    provider_api/
    local_llm_api/
    local_llm_llamacpp/
```

```
    provider_openai_compatible/
    search_api/
    curated_search/
    encrypted_storage/
    desktop_gateway/
  native/
    speech_engine/
    speech_abi/
  schemas/
    provider_response/
    search_evidence/
    local_api/
    events/
  models/
    manifests/
    licenses/
  tests/
    safety_corpus/
    child_speech_eval/
    integration/
    end_to_end/
  tools/
    model_packager/
    diagnostic_redactor/
  docs/
    adr/
    privacy/
    release/
```

The repository SHALL pin Rust, Flutter/Dart, CMake, compiler, ONNX Runtime, and model-tool versions. Generated interfaces SHALL be reproducible. CI SHALL build each supported target, run formatting and static analysis, execute unit/integration tests, create an SBOM, scan secrets and dependencies, verify licenses, and sign release artifacts only from protected release workflows.

16. Observability without cloud telemetry

Version 1 SHALL have no automatic analytics or crash upload. A bounded local event log MAY contain timestamps rounded where appropriate, component, non-content error code, model/provider type, durations, and resource statistics. It SHALL exclude transcripts, prompts, responses, filenames, character names, credentials, request bodies, and raw provider payloads.

Parent-triggered diagnostic export SHALL show a preview of included fields and require confirmation. Future opt-in telemetry requires a separate privacy review and shall not be introduced through a generic SDK.

17. Testing and verification strategy

17.1 Automated tests

- Unit tests for state transitions, capability selection, prompt compilation, token bounds, deletion, retention, migrations, provider eligibility/error normalization, search sanitization, and output policy.
- Property and fuzz tests for parsers, C ABI inputs, structured responses, Unicode names, audio container metadata, and local API payloads.
- Contract tests run against recorded/synthetic provider responses without containing child data.
- Integration tests for Keychain/Keystore/key wrapping, SQLCipher, model activation, cancellation, audio interruption, and desktop session authentication.

- End-to-end tests on physical reference devices for onboarding through playback.

17.2 Safety evaluation

The versioned corpus SHALL cover prompt injection, requests for secrecy, contact exchange, grooming patterns, sexual content, violence, dangerous activities, illegal activities, bullying, body image, medical advice, self-harm, abuse disclosure, emergency situations, commercial persuasion, political persuasion, identity claims, profanity, multilingual input, misspellings, homophones, and benign near-matches. Testing SHALL measure unsafe pass rate, excessive block rate, fallback quality, and age-appropriateness.

17.3 Privacy verification

- Capture all network traffic during enrollment, local conversation, cloud conversation, search, deletion, model download, and diagnostic export.
- Assert provider requests contain only allowlisted JSON fields.
- Inspect database, files, backups, logs, memory dumps where practical, and UI snapshots for secrets.
- Test delete-all and character deletion after crashes and partial operations.
- Verify local conversation requires no PlushPal or third-party domain after a verified model is installed.

17.4 Performance matrix

Benchmarks SHALL include minimum and reference iPhones, minimum and reference Android phones, Apple Silicon Mac, supported Intel Mac if retained, Windows x64, and Windows ARM64 if retained. Record model size, cold start, STT real-time factor, TTS real-time factor, peak RSS, CPU/GPU/NPU utilization, battery drain, thermal state, and 30-turn stability.

18. Requirements traceability and release gates

Gate	Required evidence
G0 Architecture	Approved ADRs for UI/core/speech boundaries, local inference, provider eligibility, curated search, desktop gateway, and key management
G1 Feasibility	Passing STT/local-LLM/TTS benchmarks, acceptable voice quality, and approved model/license manifests on each supported tier
G2 Privacy	Network allowlist tests, storage review, deletion tests, disclosure and consent review
G3 Safety	Safety corpus thresholds approved by product/safety owner; no critical open findings
G4 Security	Threat model review, mobile assessment, desktop gateway penetration test, dependency/SBOM review
G5 Beta	End-to-end physical-device matrix, accessibility pass, recovery and migration tests
G6 Store release	Signed artifacts, privacy labels/data safety forms, model licenses, support/privacy documentation

Every FR and NFR SHALL map to at least one test case in the requirements repository. Release CI SHALL reject mandatory requirements marked untested, failed, or waived without an owner, reason, and expiry.

19. Implementation plan

Phase 0: feasibility and governance

- Benchmark candidate local STT and zero-shot TTS models on reference hardware.
- Confirm redistribution and commercial-use licenses.
- Prototype Rust-to-C++ speech ABI and Flutter-to-native asynchronous event bridge.
- Validate local llama.cpp generation, device-tier selection, and experimental stateless BYOK calls without a PlushPal backend.
- Review privacy disclosures, parental consent, and voice authorization with qualified counsel.

Phase 1: vertical slice

- One character, one language, and the baseline local llama.cpp provider.
- Local enrollment, local STT, age-policy compilation, local generation, validation, local TTS, and cancellation.
- Encrypted local storage and platform credential stores.
- iOS, Android, macOS, and Windows developer builds using the same core.

Phase 2: product completeness

- Full onboarding and parental gate.
- Character management, capability-based model manager, local history policy, deletion, experimental cloud configuration, curated search, and desktop gateway hardening.
- Safety policy and regression corpus.
- Accessibility and interruption handling.

Phase 3: hardening and beta

- Performance tuning, memory/thermal degradation, migration/recovery, fuzzing, security review, privacy verification, store-compliance review, and supervised family beta.

Phase 4: release

- Signed stores/installers, documentation, model delivery, support runbook, incident response, vulnerability intake, and controlled rollout.

20. Principal risks and mitigations

Risk	Impact	Mitigation / decision gate
Voice cloning too slow or memory-heavy on older phones	Product unusable	Phase 0 benchmark; narrow device matrix; smaller model; pre-generate common phrases
Child-speech STT accuracy is poor	Broken conversations	Child-speech evaluation; push-to-talk UX; confidence retry; language scope
Prompt-only safety	Child harm/store rejection	Local output policy, provider controls,

Risk	Impact	Mitigation / decision gate
failure		regression corpus, safe fallback
Provider terms incompatible with child-directed use	Legal/release blocker	Fail-closed eligibility registry; keep cloud experimental; qualified review
BYOK setup is too complex	Low activation	Keep local mode the default; guided experimental setup and credential validation
Desktop local service attack	Secret/data exposure	Loopback authentication, origin/host checks, CSP/CORS, penetration test
Model licensing changes	Distribution blocker	Pinned model/license manifest, legal review, replaceable speech interface
Local model quality or performance varies by tier	Inconsistent UX	Capability benchmarks, signed recommendations, smaller context, tier-specific fallback, unsupported-device messaging

21. Required architecture decision records

Before production implementation passes G0, the team SHALL approve ADRs for:

16. Rust application core and C++ speech boundary.
17. Flutter UI and desktop-browser delivery.
18. Speech model selection and redistribution license.
19. Conversation-provider contract, eligibility registry, experimental BYOK, and production-cloud approval policy.
20. Encryption and platform key management.
21. Local history default and retention.
22. Safety policy implementation and release thresholds.
23. Supported OS/device/locale matrix.
24. Desktop loopback authentication protocol.
25. Model download/update mechanism per platform.
26. Curated-search sources, query minimization, untrusted-evidence handling, and cache policy.

22. Definition of done

PlushPal v1 is ready for general release only when all mandatory requirements are implemented and traced to passing tests; minimum-device speech and local-generation benchmarks pass; local mode works without conversation-network access after model installation; every external request contains only documented allowlisted fields; no prohibited local asset leaves the device; production cloud providers have current eligibility approval; credentials remain in platform stores; deletion and retention work under normal and interrupted conditions; the safety corpus meets approved thresholds across every released model/mode; accessibility and physical-device testing pass; security/privacy reviews have no unresolved critical or high-severity findings; and store declarations accurately describe search and third-party AI text processing.

Appendix A: Example minimized request

```
{
  "schema_version": 1,
  "policy_version": "child-safe-en-1",
  "age_band": "6-8",
  "character": {
    "alias": "friendly_bear_7f2a",
    "traits": ["warm", "playful", "patient"]
  },
  "recent_turns": [
    {"role": "child", "text": "Why do stars twinkle?"},
    {"role": "character", "text": "Their light wiggles as it travels through our air."}
  ],
  "current_text": "Do planets twinkle too?",
  "response_constraints": {
    "max_sentences": 3,
    "max_characters": 450,
    "format": "json"
  }
}
```

Appendix B: Reference sources

- Apple Foundation Models framework: https://developer.apple.com/documentation/foundationmodels/
- Apple App Review Guidelines: https://developer.apple.com/appstore/resources/approval/guidelines.html
- Apple App Privacy Details: https://developer.apple.com/app-store/app-privacy-details/
- Android Keystore: https://developer.android.com/privacy-and-security/keystore
- Android Gemini Nano: https://developer.android.com/ai/gemini-nano
- Google Play Asset Delivery: https://developer.android.com/guide/playcore/asset-delivery
- FTC COPPA guidance: https://www.ftc.gov/business-guidance/resources/complying-coppa-frequently-asked-questions
- FTC 2025 COPPA amendments summary: https://www.ftc.gov/news-events/news/press-releases/2025/01/ftc-finalizes-changes-childrens-privacy-rule-limiting-companies-ability-monetize-kids-data
- Sherpa-ONNX documentation: https://k2-fsa.github.io/sherpa/onnx/index.html
- ZipVoice repository: https://github.com/k2-fsa/ZipVoice
- llama.cpp repository: https://github.com/ggml-org/llama.cpp
- Qwen3 model family: https://huggingface.co/Qwen
- OpenAI Under 18 API Guidance: https://platform.openai.com/docs/guides/safety-checks/under-18-api-guidance
- OpenAI data controls: https://platform.openai.com/docs/guides/your-data
- Google Gemini API terms: https://ai.google.dev/gemini-api/terms
- MDN Origin Private File System: https://developer.mozilla.org/en-US/docs/Web/API/File_System_API/Origin_private_file_system
- MDN Cross-Origin-Embedder-Policy: https://developer.mozilla.org/docs/Web/HTTP/Reference/Headers/Cross-Origin-Embedder-Policy

Appendix C: Assumptions requiring confirmation

- Initial language is English; additional languages require separate speech and safety validation.
- The initial release is local-first; curated search and cloud conversation are separately parent-controlled external modes.
- Parent-supplied API keys are an experimental private-beta capability, not an automatically production-eligible child-directed cloud path.
- Production cloud use requires a provider/model/region arrangement approved for the target age range and retention policy; parent consent alone is insufficient.
- The parent has authority to provide the enrolled voice.
- Supported device minimums may be raised after Phase 0 benchmarks.
- Final privacy, child-safety, store, and biometric treatment will be reviewed by qualified legal/privacy professionals in each launch region.

Appendix D: Version 5.0 change summary

- Makes capability-selected llama.cpp models the baseline conversation path on supported iPhone, Android, macOS, and Windows devices.
- Adds signed model recommendations, runtime self-tests, last-known-good rollback, and unsupported-device behavior.
- Adds a curated web-search boundary with sanitized queries, private-network blocking, untrusted-evidence handling, and bounded local caching.
- Separates experimental parent-BYOK cloud testing from production cloud eligibility and keeps local encrypted history authoritative.
- Makes the versioned age policy enforceable before prompting/tool use and after generation for local, search-assisted, and cloud modes.